The `should` clause is made of two queries that search for products in the price range $500 to $1,000 *or* products with the phrase "4K Ultra HD" in the `overview` field. We get more results than you may expect (remember the `OR` condition):

```
  {
  ...
  "_score" : 12.059638,
  "_source" : {
    "overview" : ".. 4K Ultra HD display ...",
    "product" : "TV",
    "price" : 799,
    "brand" : "Samsung"
  }
},
{
  ...
  "_score" : 11.199882,
  "_source" : {
    "overview" : ".. 4K Ultra HD ...",,
    "product" : "TV",
    "price" : 639,
    "brand" : "Panasonic"
  }
},
{
  ...
  "_score" : 10.471219,
  "_source" : {
    "overview" : ".. 4K Ultra HD screen.. ",
    "product" : "TV",
    "price" : 1599,
    "brand" : "LG"
  }
}
...
```

The results include products that are not in the specified price range (for example, the third product in the returned result is $1,599 (way beyond what we've asked for); however, it is a match because the second criterion, `4K Ultra HD`, matched. This indicates that the `should` clause operates on the `OR` condition.

There's more than using the `OR` condition on queries when executing the search for a `should` clause. Although we only ran the `should` query, usually it's joined with other clauses like `must` and `must_not`. The advantage of using a `should` clause with a `must` clause is that the results that match the query in the `should` clause get a boosted score. Let's discuss this in detail in the next section, using an example.

BOOSTING THE SCORE WITH A SHOULD CLAUSE

The query in listing 11.8 has a `should` clause, which returns positive results using an `OR` condition. When used with the `must` clause, the `should` clause adds weight to the relevance scoring. Let's say we issue a `must` clause that matches an LG TV.